Parte 1 – Diagnóstico sem índices

Consulta	Tabela	type	key	rows	Extra
Q1	locacao	ALL	NULL	498.636	Using where
Q2	cliente	eq_ref	PRIMARY	1	Using where
Q2	locacao	ALL	NULL	498.636	–

Tempo real da Q1 (EXPLAIN ANALYZE): 94,4 ms – 500.000 linhas lidas para devolver 10.

Na Q2 o otimizador começou por `locacao` (varredura completa) e buscou cada cliente pela chave primária.

Por que o MySQL não teve alternativa a ler a tabela inteira? Não havia índice em `id_cliente` (`possible_keys` NULL) e a chave primária (`id`) não ajuda a achar as locações de um cliente. Sem uma estrutura ordenada que leve direto às linhas, a única forma de garantir que todas foram encontradas é examinar cada uma (`type=ALL`).

Parte 2 – Criando índices e comparando

Consulta	Tabela	type	key	rows
Q1	locacao	ref	idx_loc_cliente	10
Q2	cliente	ref	idx_cli_cpf	1
Q2	locacao	ref	idx_loc_cliente	9

Tempo real da Q1 agora: 0,0821 ms (antes: 94,4 ms, cerca de 1.150x mais rápido).

Seletividade na prática

Filtro	Parcela real da tabela	type	key	rows (estimado)
`status = 'cancelada'`	2%	ref	idx_loc_status	18.332
`status = 'concluida'`	74%	ref	idx_loc_status	249.318

O índice idx_loc_status existe nos dois casos. Por que o resultado do plano é diferente? O otimizador decide pelo custo estimado, que depende de quantas linhas o filtro devolve. Com `'cancelada'` (2%) o índice leva a poucas linhas e compensa. Com `'concluida'` (74%), cada entrada do índice exige outra busca na tabela, o que tende a custar mais que uma leitura sequencial completa. O esperado era o plano trocar para ALL.

Nesta execução o MySQL usou o índice nos dois casos: a estatística do índice registrou cardinalidade 2 (existem 3 valores) e estimou ∼50% da tabela em vez dos 74% reais. O EXPLAIN ANALYZE confirmou que a varredura completa seria mais rápida: 114 ms contra 386 ms com o índice.

Parte 3 – Índices que o MySQL ignora

Caso	type antes	type depois	rows antes → depois	Por que o índice não era usado?
3A	ALL	range	498.636 → 24.756	A função `DATE_FORMAT()` é aplicada a cada linha. O índice guarda `data_retirada` ordenada, não o resultado da função.
3B	ALL	range	50.243 → 111	Com % no início, o começo do texto é desconhecido e o índice (ordenado pelo primeiro caractere) não tem ponto de partida.
3C	ALL	ref	50.243 → 1	`cpf` é VARCHAR comparado com número: o MySQL converte cada `cpf` para número, e o índice de texto deixa de servir (Warning 1739).

Consultas reescritas

3A

```
SELECT * FROM locacao
WHERE data_retirada >= '2025-06-01' AND data_retirada < '2025-07-01';
```

3B

```
SELECT * FROM cliente WHERE nome LIKE 'Cliente 123%';
```

3C

```
SELECT * FROM cliente WHERE cpf = '000000004242';
```

No 3C o texto precisa dos zeros à esquerda: 4242 não encontraria o cliente.

Parte 4 – Índice composto

Consulta	type	key	rows
(a) `id_cliente = 4242 AND data_retirada >= '2025-01-01'`	range	idx_loc_cli_data	3
(b) `data_retirada` em junho/2025	range	idx_loc_data	24.756
(b) com `idx_loc_data` invisível	ALL	NULL	498.636

Na consulta (a), qual índice o MySQL escolheu (key)? `idx_loc_cli_data`. O `key_len = 7` mostra que as duas colunas foram usadas (4 bytes do INT + 3 do DATE), e a estimativa caiu para 3 linhas.

Desafio: com o índice simples de data invisível, o composto ajuda na (b)? Não. O plano voltou para ALL e o composto nem aparece em `possible_keys`. Ele é ordenado primeiro por `id_cliente` e só depois por data, então as locações de junho ficam espalhadas pelo índice (um

trecho por cliente) e não formam uma faixa contínua. Vale a regra do prefixo mais à esquerda: o filtro só na segunda coluna não aproveita o índice.

Parte 5 – Fechamento

Qual consulta teve o maior ganho com o índice e como você percebeu isso no plano? A Q1. O plano passou de `type ALL` com 498.636 linhas para `type ref` em `idx_loc_cliente` com 10 linhas, e o tempo real caiu de 94,4 ms para 0,08 ms.

Cite um caso em que existia índice e ele não foi usado, e a razão. No 3C, `idx_cli_cpf` aparecia em `possible_keys`, mas o plano foi ALL: o `cpf` (VARCHAR) foi comparado com um número e a conversão implícita inutiliza o índice. O mesmo ocorreu com o composto na consulta (b), que filtrava só a segunda coluna.

Que custo todos esses índices trazem para as operações de escrita? Cada índice é uma B-tree separada que precisa ser mantida. Um INSERT grava a linha e uma entrada em cada um dos 4 índices secundários de `locacao`; um DELETE remove todas elas; um UPDATE em coluna indexada tira e recoloca a entrada. Isso significa mais I/O, mais disco e memória e escrita mais lenta. Alguns índices ainda são redundantes: `idx_loc_cliente` é prefixo de `idx_loc_cli_data`, e `idx_loc_status` é pouco seletivo.